MV Inventario — System Health Audit (14 de marzo de 2026)

Alcance: evaluar backend (server.js + DAOs + configuraciones), capa de datos (schema SQL) y flujos frontales (settings, historial, logos) para identificar fallas, riesgos y soluciones permanentes.

1. Contexto operativo

- El backend principal es `server.js` (HTTP puro con múltiples rutas) y depende del pool `config/database.js`, que actualmente envía la zona horaria `America/Colombia` a MySQL (warning repetido en `server.log`). Hay un script completo en `sql/init-db-complete.sql` pero la carpeta `migrations` está vacía salvo la documentación. Al borrar los archivos de migración, el proceso de restaurar el esquema se reduce a ejecutar manualmente ese SQL.
- El reporte en `server.log` al arranque lista los endpoints disponibles (productos, movimientos, reportes, auth, etc.). Si el schema no existe, cualquiera de esos endpoints fallará con `ER_NO_SUCH_TABLE`.

2. Hallazgo crítico — Esquema y migraciones

Problema: No existe un mecanismo reproducible para reconstruir las tablas (usuarios, productos, configuraciones, facturas, historial) y controlar cambios. **Evidencia:** `migrations/` solo contiene `README.md`. El script real está en `sql/init-db-complete.sql`, que borra y crea tablas cada vez pero no se integra como migración incremental. **Impacto:** cualquier despliegue fresco carece de tablas como `historial_precio`, `configuracion`, `factura`, `detalle_factura`, `factura_anulacion`, `impuesto` y `secuencia_documento`. Las funcionalidades de historial, settings, logos y facturación fallan en cascada. **Solución permanente:**

- Adoptar un runner de migraciones (knex, sequelize-cli, db-migrate, umzug, etc.) o un script Node/SQL que ejecute `sql/init-db-complete.sql` y lleve un `migrations` table con versión. Incluir este paso en `npm run migrate` y en los contenedores Docker/ECS.
- Extraer la parte de `CREATE TABLE` / `INSERT` en archivos separados (01-init.sql, 02-config.sql, 03-facturas.sql) para tener historial de cambios y permitir `ROLLBACK`/auditoría.
- Incluir prechecks en el arranque (`testConnection` actual) que prueben presencia de tablas clave; si faltan, el servicio debe detenerse con una recomendación para ejecutar migraciones.

3. Historial de precios (alta prioridad)

Problema: la ruta `GET /api/productos/:id/historial-precio` existe, pero depende de la tabla `historial_precio` y los registros insertados por `HistorialPrecioDAO.registrarCambio`. Si la tabla no existe, MySQL responde `ER_NO_SUCH_TABLE` y el modal de historial se queda vacío. **Evidencia:** `HistorialPrecioDAO` (en `dao/HistorialPrecioDAO.js`) apunta a `historial_precio`; `ProductoDAO.actualizar()` llama a `registrarCambio` cuando detecta nuevos precios. Sin embargo, las migraciones perdidas eliminan la tabla entera, lo que corta el flujo completo. **Impacto:** ninguna actualización de producto deja rastro en el dashboard, el endpoint de historial devuelve error 500 y el front (`Frontend/pages/historial-precios.html`) muestra "Error al cargar historial". **Soluciones escalables:**

1. Recrear `historial_precio` (y sus índices/llaves) como parte de la migración inicial. Asegurarse de que `sql/init-db-complete.sql` o la nueva serie `migraciones/0x-historial.sql` incluya la tabla y la inyección de datos básicos.

2. Convertir `ProductoDAO.actualizar` para que exija `usuario_id` (actualmente default 1), asegurando trazabilidad por usuario y permitiendo filtrar por `usuario_nombre` en la interfaz.
3. Añadir tests de integración para el endpoint de historial (crear producto, actualizar precio, descargar historial). Un script automatizado debe limpiarse de forma segura tras cada ejecución.
4. Agregar una rutina de "seed" para llenar `historial_precio` con valores iniciales de prueba (no solo `configuracion`). Esto facilita la validación en QA.

4. Settings y configuraciones de identificación (alta prioridad)

Problema: las claves que usa la UI para mostrar el logo (`empresa.logo_path` , `empresa.logo.apply_ui` , `empresa.logo.apply_reports` , `empresa.logo_data` , `empresa.mostrar_logo` , `facturacion.pie_pagina`) no están inicializadas en el script SQL. El backend tampoco las crea antes de hacer `UPDATE` , así que las consultas devuelven `null` y las actualizaciones silenciosamente no modifican nada. **Evidencia:** `sql/init-db-complete.sql` solo inserta `empresa.logo_url` y ninguna de las otras claves. Las rutas de configuración (`/api/configuracion` , `/api/admin/configuracion`) devuelven un objeto plano; la UI `Frontend/scripts/settings.js` busca `'empresa.logo_path'` y `'empresa.logo.apply_ui'` pero recibe `undefined` . Incluso la función `uploadLogo()` del frontend extrae `result.data.path` , pero desde el backend `server.js` se responde `data: { logoPath }` , así que el preview nunca se actualiza. **Impacto:**

1. La interfaz del dashboard y navbars nunca aplican el logo personalizado porque `logo.apply_ui` no existe y la ruta no se actualiza.
2. El botón de subir logo no puede mostrar la ruta nueva (`result.data.path` es undefined) y el almacenamiento masivo no se persiste.
3. El PDF de facturas (y cualquier export Excel/PDF que use `ConfiguracionDAO.obtenerConfiguracionParaPDF`) no puede incrustar la imagen porque `empresa.logo_data` tampoco se guarda. **Soluciones permanentes:**

- Extender el seed SQL para insertar todas las claves obligatorias con valores por defecto: `empresa.logo_path` ('), `empresa.logo_data` (NULL), `empresa.logo_mime` ('), `empresa.logo.apply_ui` (1), `empresa.logo.apply_reports` (1), `empresa.mostrar_logo` (1), `facturacion.pie_pagina` ('Gracias por su compra'). Usar `INSERT ... ON DUPLICATE KEY UPDATE` para mantener integridad.
- Hacer que la ruta `/api/admin/configuracion/logo` haga: 1) `INSERT` o `UPDATE` de `empresa.logo_path` , `empresa.logo_mime` , `empresa.logo_data` (base64), 2) ajuste `empresa.logo.apply_ui` y `empresa.logo.apply_reports` , 3) responda `data: { path: logoPath }` . Más aún, incluir el digest `logo_hash` para prevenir overwritten accidental (opcional).
- Ajustar `uploadLogo()` a `result.data.logoPath` o cambiar backend para devolver `path` ; también validar que la respuesta incluya la URL absoluta (prefijo `process.env.CORS_ORIGIN` o `process.env.APP_ORIGIN`).
- Añadir validaciones de tamaño (≤80x80) en el backend y front (ya se propone en `factura.hbs`). En el backend, extraer dimensiones con `sharp` y rechazar uploads demasiado grandes.
- Crear una función reutilizable en el backend que convierta el archivo a base64 y lo escriba en `configuracion` (`empresa.logo_data`). Esta misma función puede alimentar `ConfiguracionDAO.obtenerConfiguracionParaPDF` , evitando múltiples lecturas del FS.
- Añadir una columna `categoria` llamada "Branding" y agrupar en el UI para que sea visible.

5. Logotipo en PDF/Excel/Facturación (media/alta prioridad)

Problema: el generador de PDF (`utils/generador-factura-pdf-mejorado.js` + plantilla `utils/templates/factura.hbs`) usa `logo_data` (base64) para incrustar la imagen y la marca de pie de página, pero `ConfiguracionDAO.obtenerConfiguracionParaPDF` solo devuelve `logo_data` si existe.

Además, la plantilla exige `logo_data` mientras que la UI de facturación se basa en `logo_path`. No hay proceso que convierta `/uploads/logo/` en base64 ni que repare `logo.apply_reports`. **Evidencia:** `GeneradorFacturaPDF.generarFactura` no transforma archivos de disco en base64; simplemente lee `configuracion.logo_data`. `routes/facturas.js` llama correctamente a `ConfiguracionDAO.obtenerConfiguracionParaPDF()`, pero si `logo_data` es null, no se muestra ninguna imagen. El front de facturación (e.g. `Frontend/pages/movimientos.html`) usa `empresa.logo_path` para renderizar el logo del dashboard, pero ese valor nunca se setea. **Impacto:** ni el PDF ni el dashboard ni los reportes Excel reciben el logo personalizado; la marca corporativa queda en `assets/img/logo_default.png`. Además, el endpoint `/api/admin/configuracion/logo` no limpia `uploads/logo` ni valida la extensión, lo que facilita subida de archivos maliciosos. **Soluciones escalables y seguras:**

- En el backend de uploads, después de escribir el archivo en `uploads/logo`, leerlo y convertirlo a base64 (con `fs.readFileSync(filepath, { encoding: 'base64' })`), luego guardar ese texto en `configuracion` (`empresa.logo_data`). Registrar también `empresa.logo_mime` para servirlo correctamente.
- Cambiar `ConfiguracionDAO.obtenerConfiguracionParaPDF` para que use el `logo_path` si `logo_data` no existe: leer el archivo del disco bajo `uploadDir`, convertirlo y devolverlo. Esto permite mantener compatibilidad con logos existentes.
- Eliminar logos antiguos de `uploads/logo` (por ejemplo, conservar los últimos 5). Automatizar el housekeeping (job que borra >30 días). Esto evita relleno de disco y reduce superficie para archivos maliciosos.
- Validar `imageBase64` en el backend: usar `file-type` o `sharp` para verificar que realmente sea PNG/JPEG antes de escribir.
- Paralelamente, la UI de dashboard (`Frontend/scripts/navbar.js`, `Frontend/pages/dashboard.html`) debe preguntar por `empresa.logo.apply_ui` y `empresa.logo_path`; si la clave es `true` y hay ruta, aplicar la imagen con `?t=` para evitar caché.

6. Recomendaciones generales

1. Versionar el script `sql/init-db-complete.sql` con git tags para asegurarse de poder reproducir el esquema. Al usar `docker compose up`, ejecutar `npm run migrate` (o `node scripts/migrate.js`).
2. Agregar pruebas end-to-end para: login + actualizar producto + ver historial; login + actualizar logo + generar factura PDF; login + cambiar configuraciones + verificar API. Estas pruebas ayudan a cazar regresiones.
3. Establecer políticas de saneamiento en `uploads/`: solo permitir `.png`, `.jpg`, `.jpeg`, limitar a 5 MB, y utilizar `sharp` para redimensionar a 80x80 (o 1200px con compresión). Documentar esta política en README.
4. Habilitar auditoría en la tabla `configuracion` (campos `ultima_actualizacion`, `usuario_modifica` si se necesita). Esto permite volver a versiones anteriores del logo.
5. Considerar un microservicio de "branding" que exponga una API `/api/branding/logo` con metadata (path, base64, aplica en UI, aplica en facturas) y cache en memoria para evitar múltiples lecturas de disco.

7. Pasos siguientes inmediatos

1. Re-crear las tablas faltantes ejecutando `sql/init-db-complete.sql` en el entorno de desarrollo (asegurarse de usar `DB_PORT=3307`).
2. Extender el seed de configuraciones con las claves de logo y pie de página.
3. Ajustar `/api/admin/configuracion/logo` para responder con `{ path, logoBase64 }` y persistir ambos en `configuracion`.

4. Auditar los endpoints en `server.js` que realizan `UPDATE configuracion` sin `INSERT`: reconvertir a `INSERT ON DUPLICATE KEY UPDATE` o `REPLACE`.
5. Crear un PDF final (este documento) y anexarlo al repositorio para referencia.

Documento generado automáticamente; se adjunta este mismo contenido en PDF (`system-analysis.pdf`).